

Lab 10

[valid 2024-2025]

Networking

Create an implementation of the [Hex game](#), that allows remote players to start or join games. The application will contain two parts (create a project for each one):

- The **server** is responsible with the game management and mediating the players.
- The **client** will communicate with the server, sending it *commands* such as:
 - create or join a game,
 - submit a move, etc.

The main specifications of the application are:

Compulsory (1p)

- **Create the project ServerApplication.** This will contain (at least) the classes: *GameServer* and *ClientThread*.
 - Create the class *GameServer*. An instance of this class will create a *ServerSocket* running at a specified port. The server will receive requests (commands) from clients and it will execute them.
 - Create the class *ClientThread*. An instance of this class will be responsible with communicating with a client *Socket*. If the server receives the command *stop* it will stop and will return to the client the response "Server stopped", otherwise it returns: "Server received the request ...".
 - **Create the project ClientApplication.** This will contain (at least) the class: *GameClient*.
 - Create the class *GameClient*. An instance of this class will read commands from the keyboard and it will send them to the server. The client stops when it reads from the keyboard the string "exit".
-

Homework (2p)

- Create the OOP model and implement functionalities of the game.
 - The clients will send to the server commands such as: *create game*, *join game*, *submit move*, etc.
 - The server is responsible with the game management and mediating the players.
 - The games will be played under time control (blitz) with each player having the same amount of time at the beginning of the game. If a player's time runs out, the game is lost.
-

Bonus (2p)

- Implement an AI (artificial intelligence) for the game. A human player could create a game in which the opponent is the AI, instead of another human player.
- The algorithmic level should be able to determine, at any stage of the game, if at least one path may be formed from one side to another.

Resources

- [Slides](#)
- [Custom Networking](#)
- [Remote Method Invocation \(RMI\)](#)
- [Java Networking](#)

Objectives

- Understand terms and concepts related to networking: protocol, IP, port, URL, socket, and datagram.
- Get familiar with the client-server programming model.
- Write programs that communicate with other programs on the network, using TCP or UDP.
- Get acquainted with RMI technology.